C200 Programming Assignment № 8

Dr. M.M. Dalkilic
Computer Science
School of Informatics, Computing, and Engineering

Indiana University, Bloomington, IN, USA

April 6, 2024

The HW is due on **Friday, April, 12 at 11:00 PM EST**. Please commit, push and submit your work to the Autograder before the deadline.

1. Make sure that you are **following the instructions** in the PDF, especially the format of output returned by the functions. For example, if a function is expected to return a numerical value, then make sure that a numerical value is returned (not a list or a dictionary). Similarly, if a list is expected to be returned then return a list (not a tuple, set or dictionary).

2. Test **debug** the code well (syntax, logical and implementation errors), before submitting to the Autograder. These errors can be easily fixed by running the code in VSC and watching for unexpected behavior such as, program failing with syntax error or not returning correct output.

3. Make sure that the **code does not have infinite loop (that never exits) or an endless recursion (that never completes)** before submitting to the Autograder. You can easily check for this by running in VSC and watching for program output, if it terminates timely or not.

4. Given that you already tried points 1-3, if you see that Autograder does not do anything (after you press 'submit') and waited for a while (30 seconds to 50 seconds), try refreshing the page or using a different browser.

5. Once you are done testing your code, comment out the tests i.e. the code under the __name__ == "__main__" section.

Problem 1: Root Finding with Newton-Raphson

We discussed in lecture the general problem of finding roots and how ubiquitous it is. Given a function f and interval $[a, b]$ find the $x' \in [a, b]$ (presuming it exists) such that $f(x') = 0$. We call x' a root. For example, if $f(x) = x^6 - x - 1$, $f(2) = 61$ and $f(1) = -1$. Then there must be some value $x' \in [1, 2]$ such that $f(x') = 0$. Using this approach we find

$$f(1.1347305283441975) = 6.573837356116385e - 05$$

Note that the number $6.573837356116385e - 05 = 0.00006573837356116385$ is so small that for the purpose of this problem, we can think of it as approximately equal to 0.

The Newton-Raphson is an algorithm to find roots that uses a function and its derivative to find a root. It is described recursively:

$$x_0 = estimate \tag{1}$$

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \tag{2}$$

To remind you, a derivative is a function that characterizes the way a function changes as inputs change. Equation 3 is a common approximation of the derivative we saw in the last homework. Fig. 1 shows an iteration and the approximation of the root.

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} \quad h \text{ is tiny, positive} \tag{3}$$

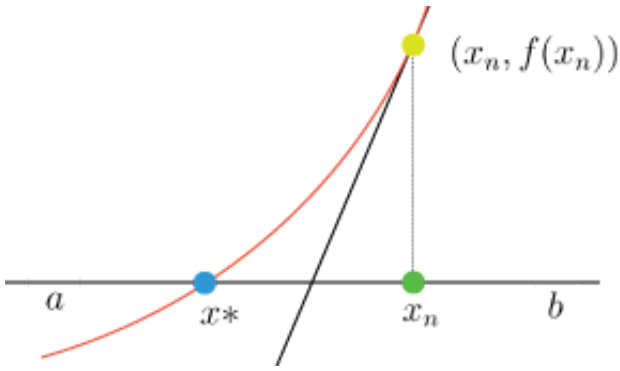

Figure 1: The root is $x*$. Our approximation x_n moves toward the root as long as we're larger than our threshold. Observe in the graphic that $f(b)$ is positive and $f(a)$ is negative insuring that there exists a root $x*$, $f(x*)$.

Another use of this approximation is to find maximal profit. Consider the following example:

$$\text{cost}(x) = 2000 + 500x \tag{4}$$

$$\text{revenue}(x) = 2000x - 10x^2 \tag{5}$$

$$\text{profit} = \text{revenue}(x) - \text{cost}(x) \tag{6}$$

You **must** observe that it is easy to write equation 6 in terms of x as: $(2000x - 10x^2) - (2000 + 500x)$. Since this is an equation, we can find the maximal profit by taking the derivative and

solving for zero (finding the root). In this case, we'll find that the maximum profit occurs when $x = 75$ (however, we want to approximate the root rather than explicitly calculating the derivative and setting it to 0).

When the program is run we have the following output:

```
f(2) = 61
f(1) = -1
f(1.1347305283441975) = 6.573837356116385e-05 ~ 0.0
x = 74.99999941508389
The maximum profit is about $54250.0
```

with the plot:

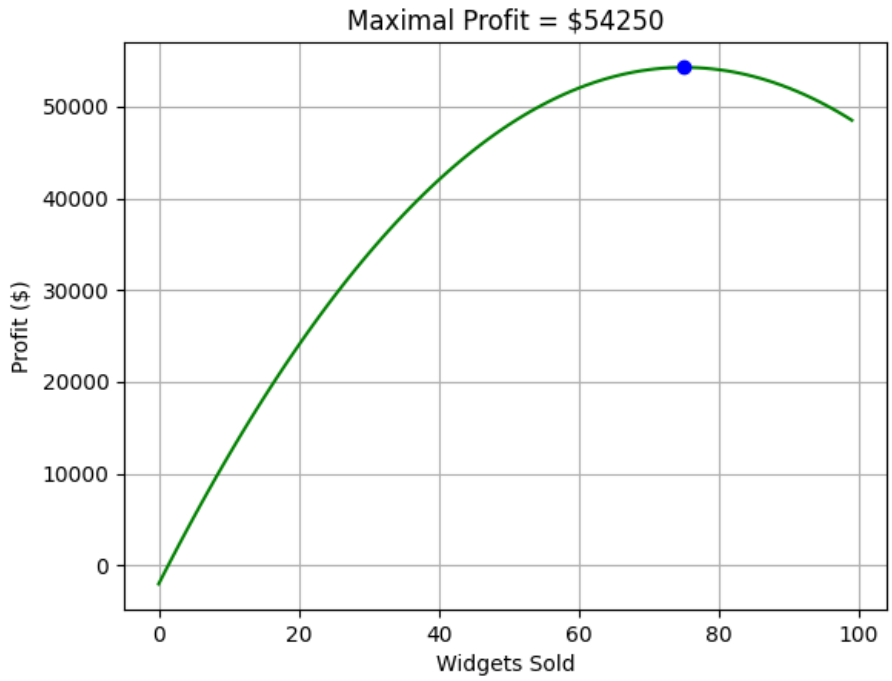

Figure 2: Using Newton-Raphson to find maximal profit $x = 75$.

Deliverables Programming Problem 1

- Complete the functions.
- You should use equation-3 for approximating the derivative.

Problem 2: Clustering with k-means, interpreting results, and visualization

Figure 3: *Versicolor* iris.

Clustering (or more formally, partitioning) is the most common AI task. We will be using scikit-learn's clustering implementation of the most commonly used algorith, k-means:
`https://scikit-learn.org/stable/modules/clustering.html`
Please read this page. In this problem, you'll cluster a collection of three kinds of irises, *Setosa*, *Versicolor*,*Virginica* based on four properties of the petal and sepal (protect the budding flower). So, in this problem, we have 150 flowers $F = \{f_0, \ldots, f_{149}\}$ each with four properties $f_i = (sl_i, sw_i, pl_i, pw_i)$ where sl is sepal length, sw is sepal width, pw is petal width, and pl is petal length. Using only these properties we want to cluster the flowers and create three blocks: $F = \{F_0, F_1, F_2\}$ where each of these blocks *should* contain only one type of iris. Because we actually know the species (not present in actual clustering), we can validate our results. The most common is the Rand Index. Please read about it here:
`https://scikit-learn.org/stable/modules/generated/sklearn.metrics.rand_score.html`
The data set F then contains 150 flowers with four properties and its actual species. We will show the task of clustering using k-means with $k = 3$ (number of blocks) on this data set and then you will implement a function to repeat the process but with only specific features (not all the features). The following code will help you to understand the steps of this process. When you implement the function, the code will laregely remain the same except few changes which we will explain after the code. Observe on line 11 that we're using all four features (or attributes).

```
1
2  #get data
3  iris = pd.read_csv('iris.csv')
4  print(iris.shape)
5  print(iris.head())
6
7  # prepare for clustering--use all the four properties
8  # leave out the actual species (we don't want to use the actual ↩
       species while clustering)
9  # Note the use iloc to subset the data and extract what we need
10 # Show correlation between all the features
11 iris_features = iris.iloc[:,[0,1,2,3]].values
12 corr = pd.DataFrame(iris.iloc[:,[0,1,2,3]]).corr()
13 print(corr)
14
15
16 # use scikit-learn and cluster the data in 3 blocks
17 result = KMeans(n_clusters=3,  random_state=0, n_init="auto")
```

```python
iris_cluster_labels = result.fit_predict(iris_features)


# Print some information
print("The cluster labels:")
print(iris_cluster_labels)
print("The cluster centers:")
print(result.cluster_centers_)
rand_index = rand_score(iris_cluster_labels,iris['species'])
print(f"The rand index is {round(rand_index,2)}")



#show how pure each block is (meaning if it contains flowers of only one type or multiple types)
species = ['setosa', 'versicolor','virginica']
dsp = { j:[]  for j in species }
for i,j in enumerate(iris_cluster_labels):
    dsp[iris.species[i]].append(j)
print("The three clusters and counts of members")
for k,v in dsp.items():
    print(f"{k} {v.count(0),v.count(1),v.count(2)}")


#plot data colored by k-means labels with actual labels side-by-side
X,Y = [i[0] for i in iris_features],[i[1] for i in iris_features]
colors = [['b','g','c'][i] for i in iris_cluster_labels]
fig, ax = plt.subplots(1, 2)
sc1 = sn.scatterplot(data=iris,x='petal_width',y='petal_length',hue=iris_cluster_labels,ax=ax[0])
sc2 = sn.scatterplot(data=iris,x='petal_width',y='petal_length',hue='species',ax=ax[1])
sc1.set(title="K-means")
sc2.set(title="Actual")
plt.show()
```

has output

```
(150, 5)
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
              sepal_length  sepal_width  petal_length  petal_width
```

```
9  sepal_length      1.000000    -0.109369      0.871754     0.817954
10 sepal_width      -0.109369     1.000000     -0.420516    -0.356544
11 petal_length      0.871754    -0.420516      1.000000     0.962757
12 petal_width       0.817954    -0.356544      0.962757     1.000000
13 The cluster labels:
14 [1 1 1 1 1 1 1 1 1 1 1 1 1
15  1 1 1 1 1 1 1 1 1 1 1 1 1
16  1 1 1 1 1 1 1 1 1 1 1 1 1
17  1 1 1 1 1 1 1 1 1 1 1 2 0
18  2 0 0 0 0 0 0 0 0 0 0 0 0
19  0 0 0 0 0 0 0 0 0 0 0 0 2
20  0 0 0 0 0 0 0 0 0 0 0 0 0
21  0 0 0 0 0 0 0 0 0 2 0 2 2
22  2 2 0 2 2 2 2 2 2 0 0 2 2
23  2 2 0 2 0 2 0 2 2 0 0 2 2
24  2 2 2 0 2 2 2 2 0 2 2 2 0
25  2 2 2 0 2 2 0]
26 The cluster centers:
27 [[5.88360656 2.74098361 4.38852459 1.43442623]
28  [5.006      3.418      1.464      0.244     ]
29  [6.85384615 3.07692308 5.71538462 2.05384615]]
30 The rand index is 0.87
31 The three clusters and counts of members
32 setosa (0, 50, 0)
33 versicolor (47, 0, 3)
34 virginica (14, 0, 36)
```

The most important scikit-learn elements are lines 17-18. Line 17 creates a k-means object. We want three blocks. The remaining two parameters are standard (read for more information). Line 18 creates an array of so-called labels. These are integers 0,1,2 that represent the block to which each f belongs. The remainder of the code shows the elements of the clustering. The quality is the Rand Index 0.87 which is good. We use seaborn (that you must install, more information is given in the starter code) that is paired with pandas. Observe in Figure-4, we have colored the flowers by their respective class labels (obtained from Kmeans clustering) on the left, and by the actual species (flower type or actual label) on the right. As you can see, the clustering appears good.

In this problem, your task is to use the correlation matrix (lines 12-13) that indicate how pairs of columns are linearly correlated. The rule of thumb is that we want data that is **not** linearly correlated (least correlated are closest to 0). To that end, you'll complete a function lst_c_2(data) that takes the original data and returns three value:

1. The first is a 2 dimensional numpy array. The values of the columns of this array are the actual feature values of the least correlated columns.

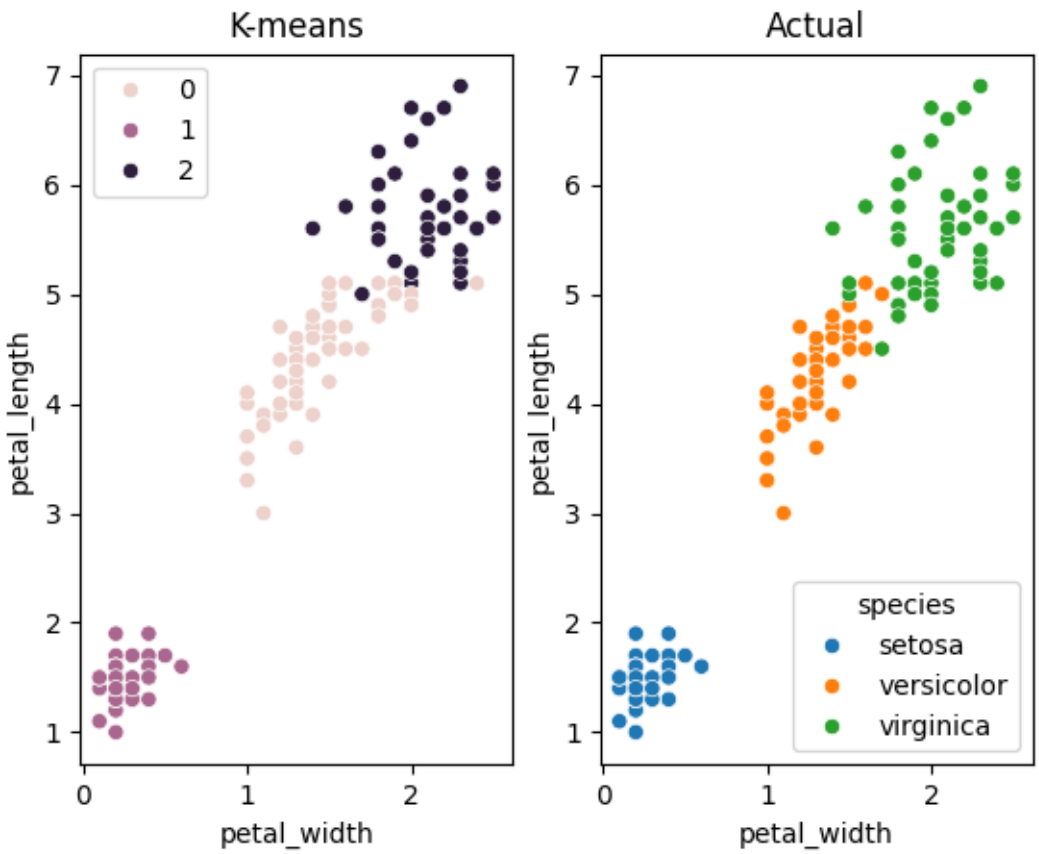

Figure 4: The left plot is k-means colored by class label. The right plot is the actual label using species. The separation between *Versicolor* and *Virginica* is the most difficult. Observe that the Rand Index is 0.87 indicating the cluster is good.

2. The second is a tuple containing the actual indices of the least correlated columns.
3. The third is the absolute value of the correlation between the columns in the pair.

To make it simple, assume that our data looks as shown after the figure on line 1-5:

```
A    B    C    D
1    2    3    4
1    3    4    5
2    1    7    9
4    5    8    10
```

Assuming that least correlated columns are 'B' and 'D' at index 1 and 3 in the data, and the correlation between them is 0.74, then the output should be as follows:

```
[[2, 4], [3, 5], [1, 9], [5, 10]], (1, 3), 0.74
```

Please ensure that you return the indices in ascending order i.e. (2, 3) or (1, 2) not (3, 2)

or (2, 1). The same is true for columns in the numpy array, if we find columsn 'B' and 'D' as least correlated then first column of numpy array should contain the values of 'B' and the second column should contain the values of 'D'.

The code is exactly the same as the original code that we explained before, except that you will implement the lst_c_2():

```
def lst_c_2(xpf):
    pass
...
features, pair_of_indices, value = lst_c_2(iris)
i,j = pair
print(f"Least corr columns: {iris.columns[i], iris.columns[j]} with {value}")
# using scikit-learn, cluster
...
```

producing

```
(150, 5)
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
Least corr columns: ('sepal_length', 'sepal_width') with 0.10936924995064931
The cluster labels:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 2 1 2 1 2 1 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2
 1 1 1 1 2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2 2 2 2 2 2 1 2 1 2 1 1 0 1 1 1 1
 1 1 2 2 1 1 1 1 2 1 2 1 2 1 1 2 2 1 1 1 1 1 2 2 1 1 1 2 1 1 1 2 1 1 1 2 1
 1 2]
The cluster centers:
[[5.00392157 3.4        ]
 [6.82391304 3.07826087]
 [5.8        2.7        ]]
The rand index is 0.82
The three clusters and counts of members:
setosa (50, 0, 0)
versicolor (0, 12, 38)
```

```
virginica (1, 34, 15)
```

and slightly different visualization (Figure-5), since you'll be clustering on only two of the features.

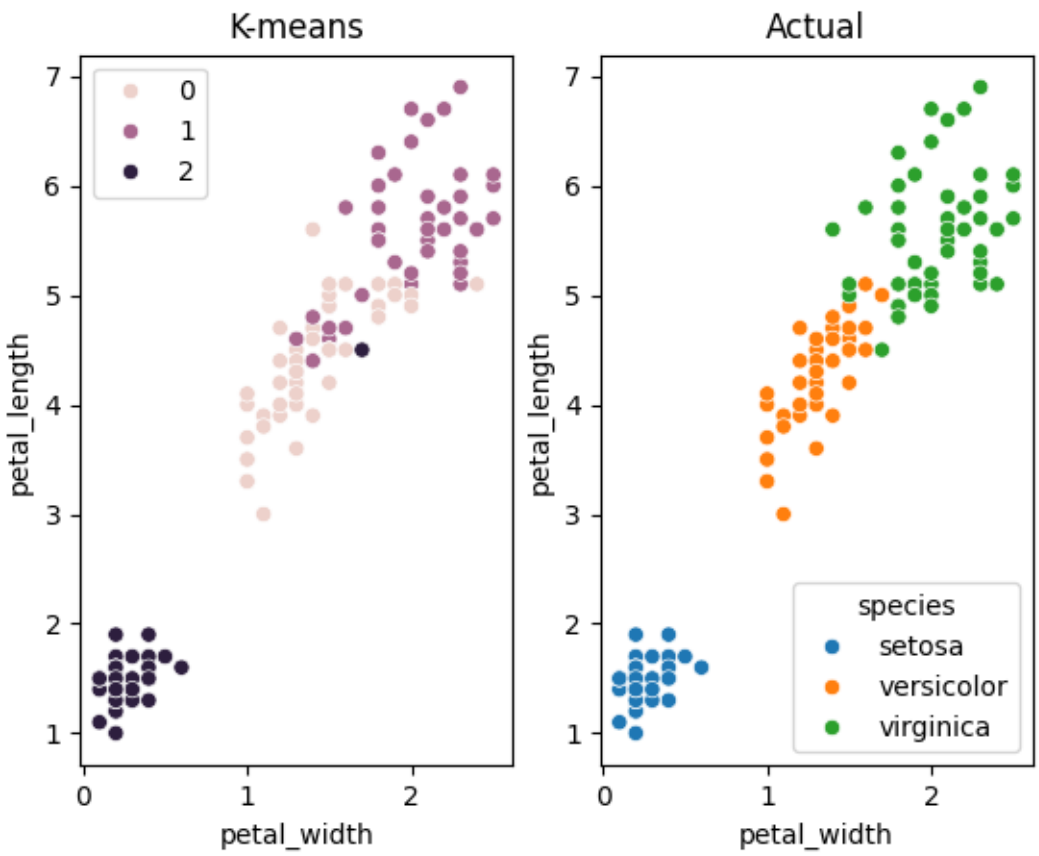

Figure 5: The left plot is k-means colored by class label using the two least correlated columns. The right plot is the actual label using species. The separation between *Versicolor* and *Virginica* is the most difficult. Observe that the Rand Index is 0.82 indicating the cluster is remains good, but a few of the flowers are further away from the block centers.

Note: Something to note is, you may or may not get the output in exacty the same order as shown. What it means is that, when one runs clustering on their system, Kmeans would still return the same cluster centers i.e. (Line 16-17), but their order may be different from what is shown. This is alright and it does not mean that your output is incorrect.

Deliverables Programming Problem 2

- You are provided with the original code to cluster under __main__.
- Read scikit-learn page on clustering to understand about input and output format and different parameters.
- Complete lst_c_2() that finds the least two correlated columns and returns an array containing values for those columns, a tuple of column IDs (as integers), and the absolute value of the correlation.

Problem 3: Simpson's Rule

In this problem, we will implement Simpson's Rule–a loop that approximates integration over an interval. Suppose we want to find the value of the integral below:

$$\int_a^b f(x)\, dx \tag{7}$$

We *could* use those pesky rules of integration–who's got time for all that, right? Or, as computer scientists, we could implement virtually all integration problems. Simpson's Rule is way of approximating an integration using parabolas (See Fig. 6). For the integration, we have to pick an even number of subintervals n and sum them up.

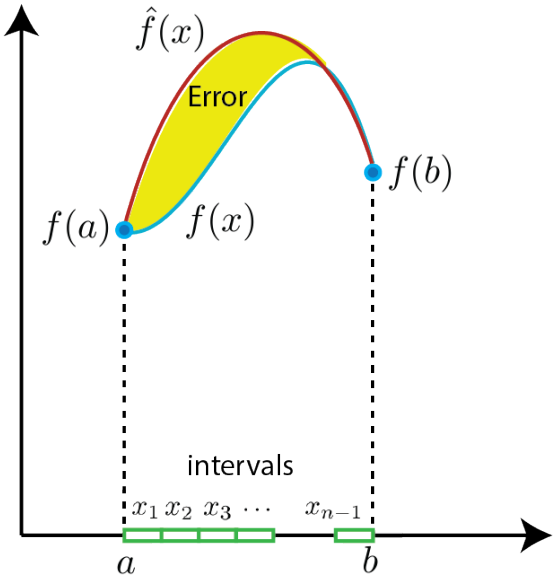

Figure 6: The function $f(x)$ integrated over a, b is approximated by $\hat{f}(x)$ using n equally sized invervals. The yellow illustrates the error of the approximation.

The *rule* is found on lines (13-14). Observe that when the index is odd that there is a coefficient of 4; when the index is even (excluding start and end, meaning $f(x0)$ and $f(xn)$ have no coefficients), the coefficient is 2.

$$\begin{aligned}
\Delta x &= \frac{b-a}{n} && (8)\\
x_i &= a + i\Delta x, \qquad i = 0, 1, 2, \ldots, n-1, n && (9)\\
x_0 &= a + 0\Delta x = a && (10)\\
x_n &= a + n\frac{b-a}{n} = b && (11)\\
\int_a^b (x)\, dx &\approx \frac{b-a}{3n}[f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + 2f(x_4) + \ldots && (12)\\
&\quad + 2f(x_{n-2} + 4f(x_{n-1}) + f(x_n)] && (13)
\end{aligned}$$

For example, the third row is the approximation to the integral

$$\int_0^\pi sin(x)\, dx \quad = \quad 2$$

using $n = 4$ intervals.

For those who want to verify,

$$\int_0^6 3t^2 + 1 = (\tfrac{1}{3})3t^3 + t\Big|_0^6 \tag{14}$$

$$= 216 + 6 = 222 \tag{15}$$

The following code:

```
data = [[lambda x:3*(x**2)+1, 0,6,2],[lambda x:x**2,0,5,6],
        [lambda x:math.sin(x), 0,math.pi, 4],[lambda x:1/x, 1, 11, 6]]

for d in data:
    f,a,b,n = d
    print(simpson(f,a,b,n))

area = simpson(lambda t: 3*(t**2) + 1,0,6,10)
t = np.arange(0.0, 10.0,.1)
fig,ax = plt.subplots()
s = np.arange(0,6.1,.1)
ax.plot(t, (lambda t: 3*(t**2) + 1)(t),'g')
plt.fill_between(s,(lambda t: 3*(t**2) + 1)(s))
ax.grid()
ax.set(xlabel ="x", ylabel=r"$f(x)=3x^2 + 1$",
title = r"Area under the curve $\int_0^6\,f(x)$ ~" + f"{round(area,2)}↩
    ")
plt.show()
```

has output:

```
222.0
41.66666666666667
2.0045597549844207
2.4491973405016885
```

and plot as shown in Figure-7.

Programming Problem 3

- Complete the functions.

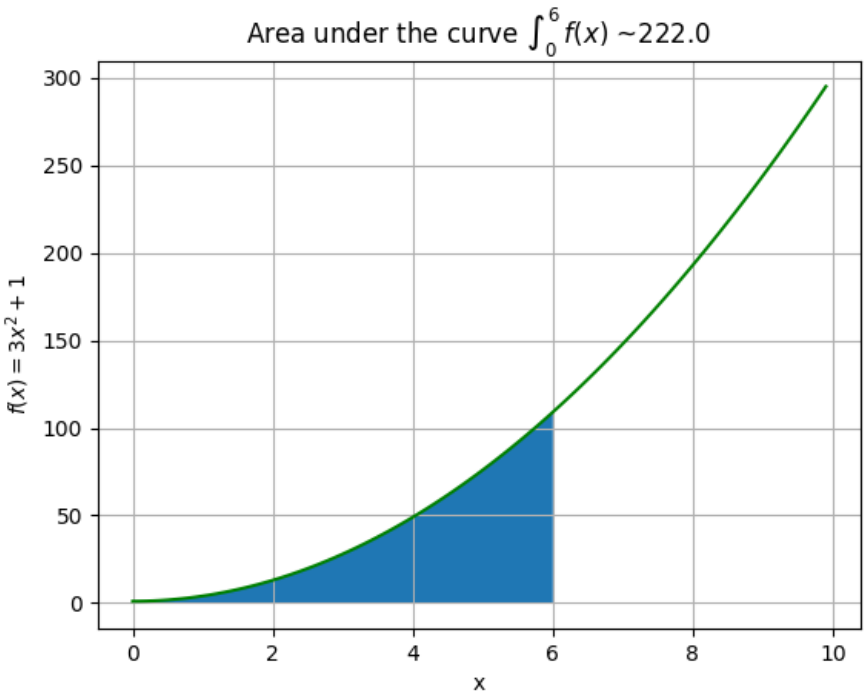

Figure 7: Using Simpson's rule to approximate an integral.

Problem 4: Central Dogma, DNA to RNA to Protein

The central dogma in biology is that DNA → RNA → protein. If you want, you can read more about it at `https://en.wikipedia.org/wiki/Central_dogma_of_molecular_biology`. In this problem you will read in a two files: The first file (amino_acids.txt) contains mapping of codons (a triplet of three DNA bases is called as **codon**) , by mapping we mean a rule that tells which specific codons will convert into a specific amino acid, and the second file (DNA.txt) contains a DNA sequence. The first file will help you to understand the mappings, and then by using the mappings, we will convert the sequence from the second file (DNA.txt) into amino acids. In essence, we will write a program to convert the DNA into protein as living organisms do! (a protein is a sequence of amino acids)

To start with, the contents of amino_acids.txt are shown below in the table.

Name	Abr.	Codons
Isoleucine	I	ATT, ATC, ATA
Leucine	L	CTT, CTC, CTA, CTG, TTA, TTG
Valine	V	GTT, GTC, GTA, GTG
Phenylalanine	F	TTT, TTC
Methionine	M	ATG
Cysteine	C	TGT, TGC
Alanine	A	GCT, GCC, GCA, GCG
Glycine	G	GGT, GGC, GGA, GGG
Proline	P	CCT, CCC, CCA, CCG
Threonine	T	ACT, ACC, ACA, ACG
Serine	S	TCT, TCC, TCA, TCG, AGT, AGC
Tyrosine	Y	TAT, TAC
Tryptophan	W	TGG
Glutamine	Q	CAA, CAG
Asparagine	N	AAT, AAC
Histidine	H	CAT, CAC
Glutamic acid	E	GAA, GAG
Aspartic acid	D	GAT, GAC
Lysine	K	AAA, AAG
Arginine	R	CGT, CGC, CGA, CGG, AGA, AGG
Stop codons	-	TAA, TAG, TGA

The first column is the full name of the amino acid. The second column is the abbreviation as one letter initial (for the same amino acid). For the Stop_codons, we use a dash. The rightmost column are what three letters of DNA are used to make the amino acid. The amino acid Arginine has an abbreviation R. There are six codon (three bases of DNA) that code for Arginine: CGT, CGC, CGA, CGG, AGA, AGG, as can be seen in the table.

About DNA.txt: It's basically a FASTA file. Please don't be confused by the name-it's just a text file that folllows certain rules, hence the special name. It has two parts: a header (information about the DNA sequence that it contains) and the DNA sequence itself. Here's the one you'll be using (don't copy them from here, we have already pushed both files to your repositories).

```
>HSGLTH1 Human theta 1-globin gene
CCACTGCACTCACCGCACCCGGCCAATTTTTGTGTT
TTTAGTAGAGACTAAATACCATATAGTGAACACCTA
AGACGGGGGGCCTTGGATCCAGGGCGATTCAGAGG
GCCCCGGTCGGAGCTGTCGGAGATTGAGCGCGCGC
GGTCCCGGGATCTCCGACGAGGCCCTGGACCCCCG
GGCGGCGAAGCTGCGGCGCGGCGCCCCCTGGAGGC
CGCGGGACCCCTGGCCGGTCCGCGCAGGCGCAGCG
GGGTCGCAGGGCGCGGCGGGTTCCAGCGCGGGGAT
GGCGCTGTCCGCGGAGGACCGGGCGCTGGTGCGCG
```

```
CCCTGTGGAAGAAGCTGGGCAGCAACGTCGGCGTCT
ACACGACAGAGGCCCTGGAAAGGTGCGGCAGGCTG
GGCGCCCCCGCCCCCAGGGGCCCTCCCTCCCCAAG
CCCCCCGGACGCGCCTCACCCACGTTCCTCTCGCAG
GACCTTCCTGGCTTTCCCCGCCACGAAGACCTACTT
CTCCCACCTGGACCTGAGCCCCGGCTCCTCACAAGT
CAGAGCCCACGGCCAGAAGGTGGCGGACGCGCTGA
GCCTCGCCGTGGAGCGCCTGGACGACCTACCCCAC
GCGCTGTCCGCGCTGAGCCACCTGCACGCGTGCCA
GCTGCGAGTGGACCCGGCCAGCTTCCAGGTGAGCG
GCTGCCGTGCTGGGCCCCTGTCCCCGGGAGGGCCC
CGGCGGGGTGGGTGCGGGGGGCGTGCGGGGCGGG
TGCAGGCGAGTGAGCCTTGAGCGCTCGCCGCAGCT
CCTGGGCCACTGCCTGCTGGTAACCCTCGCCCGGCA
CTACCCCGGAGACTTCAGCCCCGCGCTGCAGGCGTC
GCTGGACAAGTTCCTGAGCCACGTTATCTCGGCGCT
GGTTTCCGAGTACCGCTGAACTGTGGGTGGGTGGCC
GCGGGATCCCCAGGCGACCTTCCCCGTGTTTGAGTA
AAGCCTCTCCCAGGAGCAGCCTTCTTGCCGTGCTCT
CTCGAGGTCAGGACGCGAGAGGAAGGCGC
```

Though not necessary but if you want, you can read about this gene here: `https://pubmed.ncbi.nlm.nih.gov/3422341/`. In the file, the first line describes the sequence providing the name and other attributes. The remaining lines is the DNA sequence (ignore all whitespace), all lines after the header are part of the same sequence so there are no line breaks (ignore whitespaces) in the sequence.

Example: how to translate DNA into a protein

To convert from DNA to protein, we use a sequence of codons. We'll bold the protein when its translated.

Let's look at the first twelve bases: CCACTGCACTCA. Every three bases uniquely determine an amino acid.

1. Start with the first codon CCA, CCACTGCACTCA.
2. Looking at the first file we see: Proline, P, CCT, CCC, CCA, CCG. This means we can rewrite CCA as P.
3. Looking at the next codon CTG, **P**CTGCACTCA
4. We find it matches Leucine, L, CTT, CTC, CTA, CTG, TTA, TTG. So our protein is PL.
5. The next three are CAC **PL**CACTCA.
6. The table has Histidine, H, CAT, CAC.

7. The final three are TCA. **PLH**TCA

8. This matches Serine, S, TCT, TCC, TCA, TCG, AGT, AGC.

9. The protein is **PLHS**.

If you are at the end and only have two bases, you cannot match a protein, so you ignore them. Suppose we had CCAC. We know CCA is P. Then we only have C left. We ignore it.

How to approach this problem

Our main task is to take the DNA (by reading DNA.txt) and produce a string of single letters (as shown in above example, points 1-9) that reflect the encoding. Here is one way to solve this (you should implement how you feel most comfortably)

1. First you'll read in the table from amino_acids.txt, and create a dictionary whose entries are:

$$aa_d \quad = \quad \{(c_0, c_1, \ldots, c_n) : [name, letter], \ldots\}$$

where c_i is a three letter codon, $name$ is the full name of the amino acid, and $letter$ is the single letter for the amino acid. Make sure you follow the format of keys and values exactly as shown here. The function get_amino_acid() takes the file name and returns a dictionary. The dictionary is also shown below in the code listing.

2. Read in the DNA sequence, the function get_DNA() takes a file name and returns a faste data structure [header, DNA] (FASTA data structure) where header is the first line of the file DNA.txt and DNA is the DNA sequence (the sequence of A,T,G,C after the first line) (ignoring any whitespace). The read-in FASTA sequence is also shown below in the code listing.

3. The function translate() takes a FASTA data structure (that you created in step-2) and returns a string that is the translation using the dictionary (that you created in step-1). We can do a simple print to see whether our translation is the same as actual. An example of translate() function is also shown below in the code listing (you are encouraged to cross-check via pen and paper).

This code creates the dictionary and FASTA file (as a list), translates, and validates:

```
print("Dictionary")
print(aa_d)
print("FASTA file")
print(DNA_d)
print("Translations match:", str(protein == actual))

#should return "PLHS"
print(translate(["nothing", "CCACTGCACTCA"]))
```

```python

#should return "D-"
print(translate(["nothing", "GACTAA"]))
```

has output:

```
Dictionary
{('ATT', 'ATC', 'ATA'): ['Isoleucine', 'I'], ('CTT', 'CTC', 'CTA', 'CTG', 'TTA', 'TTG'): ['Leucine', 'L'], ('GTT', 'GTC', 'GTA', 'GTG'): ['Valine', 'V'], ('TTT', 'TTC'): ['Phenylalanine', 'F'], ('ATG',): ['Methionine', 'M'], ('TGT', 'TGC'): ['CYSteine', 'C'], ('GCT', 'GCC', 'GCA', 'GCG'): ['Alanine', 'A'], ('GGT', 'GGC', 'GGA', 'GGG'): ['Glycine', 'G'], ('CCT', 'CCC', 'CCA', 'CCG'): ['Proline', 'P'], ('ACT', 'ACC', 'ACA', 'ACG'): ['Threonine', 'T'], ('TCT', 'TCC', 'TCA', 'TCG', 'AGT', 'AGC'): ['Serine', 'S'], ('TAT', 'TAC'): ['Tyrosine', 'Y'], ('TGG',): ['Tryptophan', 'W'], ('CAA', 'CAG'): ['Glutamine', 'Q'], ('AAT', 'AAC'): ['Asparagine', 'N'], ('CAT', 'CAC'): ['Histidine', 'H'], ('GAA', 'GAG'): ['Glutamic_acid', 'E'], ('GAT', 'GAC'): ['AsparTic acid', 'D'], ('AAA', 'AAG'): ['Lysine', 'K'], ('CGT', 'CGC', 'CGA', 'CGG', 'AGA', 'AGG'): ['Arginine', 'R'], ('TAA', 'TAG', 'TGA'): ['Stop_codons', '-']}
FASTA file
['>HSGLTH1 Human theta 1-globin gene', 'CCACTGCACTCACCGCACCCGGCCAATTTTTGTGTTTTTAGT
AGAGACTAAATACCATATAGTGAACACCTAAGACGGGGGGC
CTTGGATCCAGGGCGATTCAGAGGGCCCCGGTCGGAGCTGT
CGGAGATTGAGCGCGCGCGGTCCCGGGATCTCCGACGAGGC
CCTGGACCCCCGGGCGGCGAAGCTGCGGCGCGGCGCCCCCT
GGAGGCCGCGGGACCCCTGGCCGGTCCGCGCAGGCGCAGCG
GGGTCGCAGGGCGCGGCGGGTTCCAGCGCGGGGATGGCGCT
GTCCGCGGAGGACCGGGCGCTGGTGCGCGCCCTGTGGAAGA
AGCTGGGCAGCAACGTCGGCGTCTACACGACAGAGGCCCTG
GAAAGGTGCGGCAGGCTGGGCGCCCCCGCCCCCAGGGGCCC
TCCCTCCCCAAGCCCCCCGGACGCGCCTCACCCACGTTCCTC
TCGCAGGACCTTCCTGGCTTTCCCCGCCACGAAGACCTACTT
CTCCCACCTGGACCTGAGCCCCGGCTCCTCACAAGTCAGAGC
CCACGGCCAGAAGGTGGCGGACGCGCTGAGCCTCGCCGTGG
AGCGCCTGGACGACCTACCCCACGCGCTGTCCGCGCTGAGC
CACCTGCACGCGTGCCAGCTGCGAGTGGACCCGGCCAGCTT
CCAGGTGAGCGGCTGCCGTGCTGGGCCCCTGTCCCCGGGAG
GGCCCCGGCGGGGTGGGTGCGGGGGGCGTGCGGGGCGGGT
GCAGGCGAGTGAGCCTTGAGCGCTCGCCGCAGCTCCTGGGC
CACTGCCTGCTGGTAACCCTCGCCCGGCACTACCCCGGAGAC
TTCAGCCCCGCGCTGCAGGCGTCGCTGGACAAGTTCCTGAGC
CACGTTATCTCGGCGCTGGTTTCCGAGTACCGCTGAACTGTG
GGTGGGTGGCCGCGGGATCCCCAGGCGACCTTCCCCGTGTTTG
```

```
AGTAAAGCCTCTCCCAGGAGCAGCCTTCTTGCCGTGCTCTCTC
GAGGTCAGGACGCGAGAGGAAGGCGC']
Translations match: True
PLHS
D-
```

Deliverables Problem 4

- Carefully read the instructions in the starter file a9.py. You may not be able to run it directly in VSC, so follow the instructions in the starter code.
- Complete the functions get_DNA(), get_amino_acids() and translate() as per the specifications.
- You are allowed to use replace for space and "{''. Other uses of it will actually be more difficult.
- For this problem, using pandas makes the autograder a little crazy–so we're eschewing dataframes. Furthermore, everything is text.

Problem 5: Marginal Cost

When producing something, the cost typically varies. The **marginal cost** is the cost incurred by producing an additonal unit of product or service. This is the derivative in disguise. Given a function $cost(x)$, we can determine the derivative as:

$$\frac{d\,cost(x)}{dx} \approx \lambda x : \frac{cost(x+h) - cost(x-h)}{2h} \tag{16}$$

for a very small value of h. For this problem assume the cost function is:

$$cost(x) = 0.0001x^3 - 0.08x^2 + 40x + 5000 \tag{17}$$

The following code:

```
U,C = [],[]
for unit in range(200,650,50):
    U.append(unit)
    mc = round(marginal_cost(cost)(unit),0)
    C.append(mc)
    print(f"For {unit} marginal cost is {mc}")
plt.plot(U,C,'b-')
plt.plot(300,round(marginal_cost(cost)(300)),'ro')
plt.xlabel("Units of Production")
plt.ylabel("Cost $")
plt.title(r"Marginal cost Cost(x) =  $0.0001x^3 - 0.08x^2 + 40x + 5000$")
plt.show()
```

has the output:

```
For 200 marginal cost is 20.0
For 250 marginal cost is 19.0
For 300 marginal cost is 19.0
For 350 marginal cost is 21.0
For 400 marginal cost is 24.0
For 450 marginal cost is 29.0
For 500 marginal cost is 35.0
For 550 marginal cost is 43.0
For 600 marginal cost is 52.0
```

and plot:

Deliverables Problem 5

- We include the cost function for you.
- This is a very short homework problem, since you've already implemented the derivative function earlier!

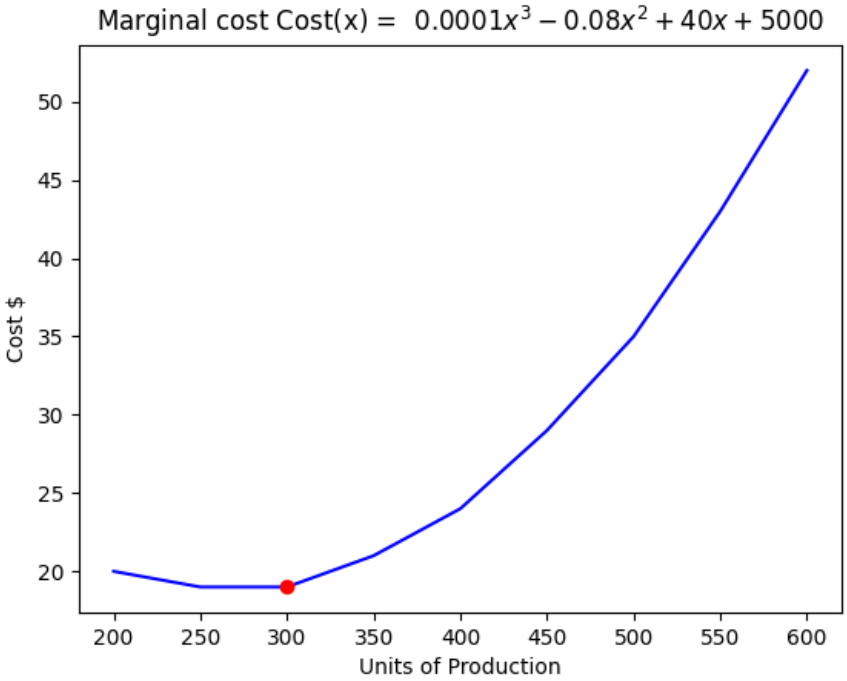

Figure 8: Marginal cost for sales 200-600 units. Observe the cost increases at 300.

Problem 6: Using a Model that Breaks

Newton's Law of Cooling gives the rate of a cooling of an object proportional to its current temperature and the ambient temperature. The model described is:

$$Temp(t) \quad = \quad (T_0 - T_a)e^{-kt} + Ta \tag{18}$$

where T_0 is the object's initial temperature, T_a is the ambient (or background) temperature, t is time elapsed, the k and experimentally determined constant. Assume $k = 0.19617$. Implement a function determine_t(T,T0,Ta,k) that, when given a final temperature T, determines the time it takes to reach that temperature. For example, this code

```
k = 0.19617
T0 = 200
Ta = 72
T = 85
print(determine_t(T,T0,Ta,k))
```

which has the initial temperature at 200°F, ambient temperature at 72°F, and final temperature at 85°F produces

```
11.66
```

Temp	Time
75	19.13
74	21.2
73	24.73

which means it will take about 11 min. 40 sec. to cool. The problem with this model is that we can't use it directly to ask how long will it take to cool to room temperature:

$$T_a = (T_0 - T_a)e^{-kt} + T_a \tag{19}$$

$$e^{-kt} = 0 \tag{20}$$

We can't take the log of 0. We confirm with Python:

```
>>> import math
>>> math.log(0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: math domain error
>>>
```

The reason is that this is a limit (convergence):

$$\lim_{t\to\infty} e^{-kt} = 0 \tag{21}$$

Indeed, if we run our solution we'll raise the same error:

```
11.66
Traceback (most recent call last):
  File "newtonlawcool", line 31, in <module>
    print(determine_t(Ta,T0,Ta,k))
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "newtonlawcool", line 13, in determine_t
    temp_change = math.log((T - Ta)/(T0 - Ta))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: math domain error
```

How can we solve this problem? We can form the linear regression over, say, the last three points:

The code uses the last three whole temperatures (we'll need our original function to determine them). We can then use numpy's polyfit: `https://numpy.org/doc/stable/reference/generated/numpy.polyfit.html` that has how to find the coefficients $y = mx + b$ as an array and how to automatically build the function from the coefficients.

```
k = 0.19617
```

```
T0 = 200
Ta = 72
T = 85
print(determine_t(T,T0,Ta,k))
print(final_temp(T0,Ta,k))
```

produces

```
11.66
27.29
```

Deliverables Problem 6

- Implement the two functions.
- Round to two decimal places.

Problem 7: String Sequence Sums

(String Sequence Sums). Suppose you have a string of digits $s_0 s_1 s_2 \cdots s_N$. Consider some contiguous subsequence $t_i t_{i+1} \cdots t_m$. This is a string sequence sum (SSS) when

$$\text{int}(t_i \cdots t_k) + \text{int}(t_{k+1} \cdots t_\ell) = \text{int}(t_{\ell+1} \cdots t_m) \tag{22}$$

For example, if t = "11235813" has:

$$1 + 1 = 2 \tag{23}$$

$$\text{int}(\text{t}[0:1]) + \text{int}(\text{t}[1:2]) = \text{int}(\text{t}[2:3])$$

$$1 + 2 = 3 \tag{24}$$

$$2 + 3 = 5 \tag{25}$$

$$3 + 5 = 8 \tag{26}$$

$$5 + 8 = 13 \tag{27}$$

The function n(xstr) that takes a digital string returns all the SSS. Additionally, if $a + b = c$ is returned, then either $a + b = c$ or $b + a = c$ are *not* included. The code

```
data = ["11235813","1111223","1111213","11121114"]
for d in data:
    print(n(d))
```

produces:

```
[['1', '1', '2'], ['1', '2', '3'], ['2', '3', '5'], ['3', '5', '8'], ['5', '8', '13']]
[['1', '11', '12'], ['11', '11', '22'], ['11', '12', '23'], ['1', '1', '2']]
[['1', '11', '12'], ['1', '1', '2'], ['1', '12', '13'], ['11', '2', '13'], ['2', '1', '3']]
[['1', '1', '2']]
```

Deliverables Problem 7

- Implement the function.
- You are free to implement this with or without recursion. Without recursion is simpler. There is a hint buried in the description of the problem.

Problem 8: Toward Monte Carlo

Most real problems cannot be solved *deterministically*. Simply, we can't predict the future. Symbolically, if we are in some state s of truth, an infinite number of states $s_1, s_2, \ldots, s_N, \ldots$ can follow. To tackle this problem we first limit the possible state to a manageable number, say $N = 4$, but this still doesn't help us *predict* which of the four will most likely be true. *Stochastic* modeling takes into account multiple, possible futures by associating each with a *probability*, a number between and included zero to one. In fact, a discrete (mass) probability is simply a mapping function from a set of states $s_1, s_2, \ldots, s_N$ to real values:

$$s_i \mapsto p_i \tag{28}$$

$$\sum_{i=1}^{N} p_i = 1 \tag{29}$$

$$\forall i, 1 \leq i \leq N \rightarrow p_i \geq 0 \tag{30}$$

The notion is that if we could rerun our history starting from s and ending in either s_1, s_2, s_3, or s_4, the relative frequency would match the probability. One of the most ubiquitous stochastic models used in Monte Carlo. We'll dive deeper into how its works and its application, but for now, we focus on preparing it. Specifically we have to find a way to use a uniform distribution (all the futures are equally like) with our data.

Suppose we have a table of growth rates of our businesses from the last five years: The

Year	YoY GR%
2019	6
2020	4
2021	8
2022	2
2023	5

first task is to normalize the data. This means we convert the numbers into probabilities (or fractions) $p \in [0, 1]$ that indicate the relative size. An outcome is that this produces a probability mass function. Let's have data = [6,4,8,2,5]. We create a probability for each value by dividing it by the sum. To create the cumulative distribution that allows us to use a uniform distribution we combine subsequent years:

Year	YoY GR%	p
2019	6	0.24
2020	4	0.16
2021	8	0.32
2022	2	0.08
2023	5	0.2

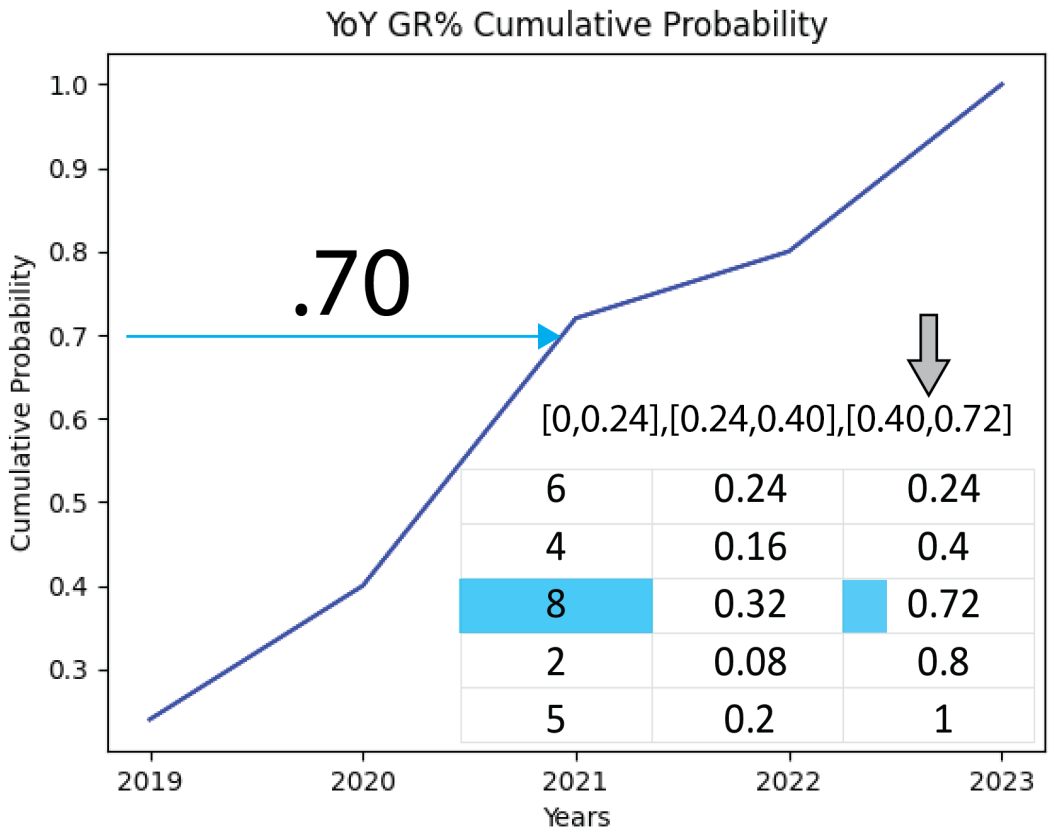

Figure 9: The cumulative distribution. Observe that picking a random value between [0,1] then finding the corresponding x value interval gives a probability.

$$2019 - 2019 = p_1 = 0.24 \tag{31}$$

$$2019 - 2020 = p_1 + p_2 = 0.4 \tag{32}$$

$$2019 - 2021 = p_1 + p_2 + p_3 = 0.72 \tag{33}$$

$$2019 - 2022 = p_1 + p_2 + p_3 + p_4 = 0.80 \tag{34}$$

$$2019 - 2023 = p_1 + p_2 + p_3 + p_4 + p_5 = 1 \tag{35}$$

Figure 9 shows how we'll use a uniform distribution to determine the original probability. We pick a value in $[0, 1]$, say, 0.70. This maps to the interval between 0.40 and 0.72. That then is year 2021 at 8%. Given a list of numbers, build a cumulative distribution and set of intervals that Monte Carlo needs. Apply this to the data in the table in the running example. The table is data = [(2019,6),(2020,4),(2021,8),(2022,2),(2023,5)] and the results will be

```
[((0, 0.24), 6), ((0.24, 0.4), 4), ((0.4, 0.72), 8), ((0.72, 0.8), 2),↩
    ((0.8, 1.0), 5)]
```

where each member is tuple ((x,y),z) where (x,y) is the interval that z occurs. The probabilities should be rounded to two places.

Deliverables Problem 8

- Implement the function.
- Round to two decimal places.

Extra Credit: Polynomial Interpolation

None of the functions demonstrated are global except build_polynomial. I'm showing how some of them work on data–but they all are local. This problem is only counted as extra credit. To that end, it can involve more complexity and should only be attempted if the other problems are completed. Most curves we encounter can be effectively modeled by polynomials. We've seen numpy's approach (we'll talk about it more in a subsequent homework), but there exists an older technique: Newton's Binomial Interpolation Formula. Understanding the theory isn't necessary to understand the formula, but it does include an extension of the binomial formula $\binom{i}{j}$ for $i, j = 0, 1, 2, \ldots$. In this case the expression:

$$\binom{x}{0} = \tfrac{1}{0!}1 \tag{36}$$

$$\binom{x}{1} = \tfrac{1}{1!}x \tag{37}$$

$$\binom{x}{2} = \tfrac{1}{2!}(x-1)x \tag{38}$$

$$\binom{x}{3} = \tfrac{1}{3!}(x-2)(x-1)x \tag{39}$$

$$\binom{x}{n} = \tfrac{1}{n!}(x-(n-1))(x-(n-2))\cdots(x-1)x \tag{40}$$

for a variable x and integer n. The approach uses a difference table which we'll discuss next. An difference operator Δ, for a list of values $\ell = [y_0, y_1, \ldots, y_n]$ produces a new list $\Delta(\ell) = [y_1 - y_0, y_2 - y_1, \ldots, y_n - y_{n-1}]$. If we write the exponent as the number of times Δ is applied, we have a difference table is:

$$dtable(\ell) = [\Delta^0(\ell), \Delta^1(\ell), \Delta^2(\ell), \ldots, \Delta^i(\ell)], i = \text{len}(\ell) - 1 \tag{41}$$

for list $\text{len}(\ell) > 1$. Assume $\ell = [-1, 0, 5, 20]$. The table can be visualized as:

ℓ	$\Delta(\ell)$	$\Delta^2(\ell)$	$\Delta^3(\ell)$
-1			
	1		
0		4	
	5		6
5		10	
	15		
20			

The code confirms this:

```
lst = [-1,0,5,20]
print(dtable(lst)) #function that takes a list and returns a difference table
```

produces:

```
[[-1, 0, 5, 20], [1, 5, 15], [4, 10], [6]]
```

We need the first value in each list. Let $\rho = [\ell[0], \Delta(\ell)[0], \Delta^2(\ell)[0], \ldots, \Delta^i(\ell)[0]]$. For this example,

$$\rho = [-1, 1, 4, 6]$$

An interesting relationship arises:

$$\ell[-1] = \rho_0 + \binom{i}{1}\rho_1 + \binom{i}{2}\rho_2 + \cdots + \binom{i}{i}\rho_i \tag{42}$$

(we're using Python for left variable) where $i = \text{len}(\ell) - 1$. In our example,

$$20 = \binom{3}{0}(-1) + \binom{3}{1}1 + \binom{3}{2}4 + \binom{3}{3}6 \tag{43}$$

$$= -1(1) + 3(1) + 3(4) + 6 = 20 \tag{44}$$

The code:

```
lst = [-1,0,5,20]

print(rho(lst))

i = len(lst) - 1
s_ = [math.comb(i,j)*r_ for j,r_ in enumerate(rho(lst))]
print(lst[-1], sum(s_))
```

produces

```
[-1, 1, 4, 6]
20 20
```

Let $D = \{(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)\}$. Then we can build a polynomial that describes D:

$$I(D) = \binom{x}{0}\rho_0 + \binom{x}{1}\rho_1 + \binom{x}{2}\rho_2 + \cdots + \binom{x}{i}\rho_i \tag{45}$$

where $i = n - 1$ and ρ_i are values from the difference table generated from the y values as before. For example, let $D = [(0, -2), (1, 5), (2, 7), (3, 10)]$. Then:

$$I(D) = -2 + 7\binom{x}{1} - 5\binom{x}{2} + 6\binom{x}{3} \tag{46}$$

$$= -2 + 7x - \tfrac{5}{2}x(x-1) + x(x-1)(x-2) \tag{47}$$

$$= x^3 - \tfrac{11}{2}x^2 + \tfrac{23}{2}x - 2 \tag{48}$$

We don't need to simplify, but do it to write a simple function to plot. The following code:

```
D = [(0,-2),(1,5),(2,7),(3,10)]
pf = lambda x:x**3 - (11/2)*(x**2) + (23/2)*x - 2
f = build_polyonomial(D)
for i in range(7):
    print(i,pf(i),f(i))
x = np.linspace(0,6,100)
X,Y = [x for x,_ in D],[y for _,y in D]
plt.plot(X,Y,'ro')
plt.plot(x,f(x),'b-')
plt.title("Newton Binomial Interpolation Formula")
plt.show()
```

produces:

```
0 -2.0 -2.0
1 5.0 5.0
2 7.0 7.0
3 10.0 10.0
4 20.0 20.0
5 43.0 43.0
6 85.0 85.0
```

and plot Line 3 sends the data to a function build_polynomial that returns the interpolated poly-

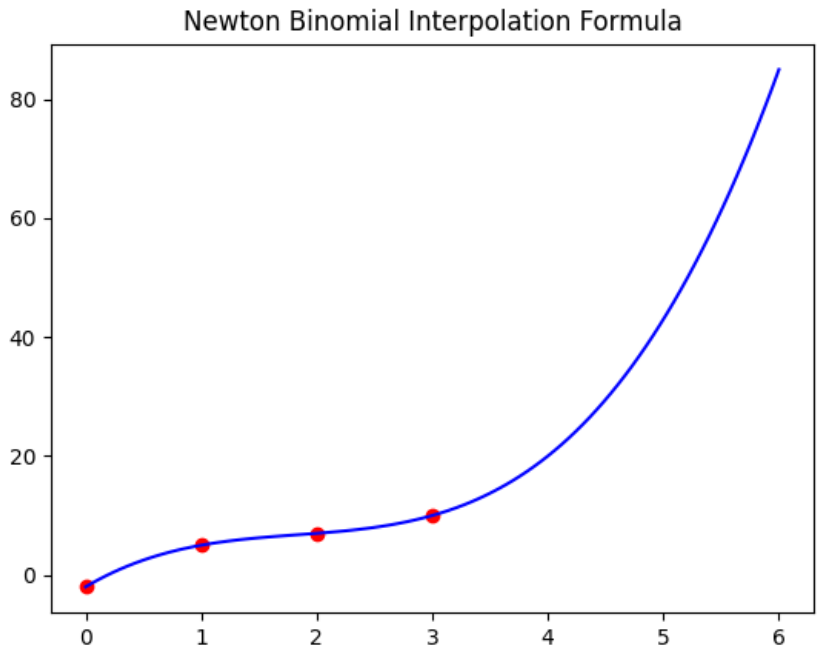

Figure 10: The points points are the data D = [(0,-2),(1,5),(2,7),(3,10)]. The blue curve the fitted polynomial.

nomial as a lambda function. Observe this function is the same as Eq. (31) and on line 2.

Deliverables Extra Credit

- Complete the build_polynomial function
- There are many approaches to building the polynomial (once the difference table is built). One approach is to treat the functional binomial $\binom{x}{n}$ as a λ function. You'll likely have to build a function (as a string) and then use eval to make it useable.
- All functions used in this problem are local to build_polynomial.
- If you build a test case, use $D = [(0, y_0), (1, y_1), \ldots, (k, y_k)])$ where successive x values differ by only 1 and start at zero. This is done to simplify the algorithm. These can be changed in a more general solution.

Programming pairs

blberger@iu.edu, andmuchi@iu.edu
bcabral@iu.edu, jmatczak@iu.edu
jakedeck@iu.edu, zstopher@iu.edu
cbremner@iu.edu, frvisco@iu.edu
lineri@iu.edu, peterant@iu.edu
langgrav@iu.edu, javitto@iu.edu
nabatja@iu.edu, brshelby@iu.edu
balian@iu.edu, jusshah@iu.edu
ocrowder@iu.edu, weinstab@iu.edu
sjbarry@iu.edu, crschuit@iu.edu
andhodge@iu.edu, snlnu@iu.edu
adevill@iu.edu, bpovinel@iu.edu
pkurzawa@iu.edu, nmetlapa@iu.edu
jkchaura@iu.edu, lawlin@iu.edu
wchorich@iu.edu, kammich@iu.edu
zcarmich@iu.edu, cmm23@iu.edu
nsfried@iu.edu, wagnebr@iu.edu
auconner@iu.edu, amatariy@iu.edu
shbivens@iu.edu, aquintas@iu.edu
harrisdg@iu.edu, sparab@iu.edu
renahan@iu.edu, wangjinz@iu.edu
craftdc@iu.edu, olipfing@iu.edu
berrynm@iu.edu, ishetty@iu.edu
chamblia@iu.edu, wardjohn@iu.edu
njustin@iu.edu, ymarrakc@iu.edu

sebcont@iu.edu, abbalokh@iu.edu
harmonnc@iu.edu, ctsvetan@iu.edu
kespejo@iu.edu, njvinson@iu.edu
marbelli@iu.edu, joprayit@iu.edu
careyjo@iu.edu, rahkpate@iu.edu
chodgman@iu.edu, ashahi@iu.edu
ecabebe@iu.edu, haoztian@iu.edu
belleli@iu.edu, breramer@iu.edu
burkhel@iu.edu, ptapiaro@iu.edu
nkushnir@iu.edu, jamread@iu.edu
cdinman@iu.edu, camyee@iu.edu
ryandaun@iu.edu, ap59@iu.edu
sambereb@iu.edu, colbstap@iu.edu
adeglur@iu.edu, jianlou@iu.edu
aadkhan@iu.edu, archshar@iu.edu
ecanant@iu.edu, katepate@iu.edu
anudivek@iu.edu, smunikun@iu.edu
conkeen@iu.edu, menonvi@iu.edu
chenkay@iu.edu, twirges@iu.edu
ankotel@iu.edu, jomalek@iu.edu
lasdodd@iu.edu, mackeyr@iu.edu
cfahy@iu.edu, jlopezmo@iu.edu
ykassahu@iu.edu, bsai@iu.edu
cjduffie@iu.edu, pratmodi@iu.edu
mbadwe@iu.edu, mgstack@iu.edu
cedonica@iu.edu, iopfings@iu.edu
jailjohn@iu.edu, corrosar@iu.edu
jvgottum@iu.edu, msolomie@iu.edu
beckab@iu.edu, shmulla@iu.edu
forlin@iu.edu, jocpoiri@iu.edu
rcerauli@iu.edu, auspiege@iu.edu
jdemian@iu.edu, reddysk@iu.edu
kimyouh@iu.edu, trimedio@iu.edu
tkonkim@iu.edu, traepena@iu.edu
sgudapur@iu.edu, mvsimi@iu.edu
arkhal@iu.edu, gstogdil@iu.edu
reafrank@iu.edu, etmillho@iu.edu
changali@iu.edu, mxrobins@iu.edu
rachguo@iu.edu, rimaeda@iu.edu
samjimen@iu.edu, arstover@iu.edu
okorkmaz@iu.edu, vupatel@iu.edu

jhbruce@iu.edu, gravipat@iu.edu
ejbusch@iu.edu, nrimawi@iu.edu
blevidut@iu.edu, benserna@iu.edu
cgreywal@iu.edu, shubtiwa@iu.edu
pafrost@iu.edu, bfmark@iu.edu
zlangley@iu.edu, johnsanc@iu.edu
jamlian@iu.edu, saravula@iu.edu
shaagore@iu.edu, smremmer@iu.edu
reychaud@iu.edu, joebrahm@iu.edu
askaul@iu.edu, mitcchar@iu.edu
hedavid@iu.edu, ggriorda@iu.edu
daveanu@iu.edu, adpulipa@iu.edu
anchinna@iu.edu, arangi@iu.edu
gmhowell@iu.edu, xsoto@iu.edu
camehugh@iu.edu, tmukund@iu.edu
nkhai@iu.edu, leoett@iu.edu
agunde@iu.edu, rsreedh@iu.edu
madimond@iu.edu, avaweish@iu.edu
cc96@iu.edu, khwillar@iu.edu
akumares@iu.edu, ceprenti@iu.edu
nbkotur@iu.edu, camlueck@iu.edu
sdharmmi@iu.edu, ksreejit@iu.edu
presharr@iu.edu, bwmoore@iu.edu
mgrass@iu.edu, syeshala@iu.edu
achitred@iu.edu, tiffyou@iu.edu
markdeng@iu.edu, tryerrag@iu.edu
allentp@iu.edu, surehma@iu.edu
allennam@iu.edu, nsmurthy@iu.edu
jaxkatz@iu.edu, nicmill@iu.edu
boding@iu.edu, nsenthi@iu.edu
emiellis@iu.edu, amtsmith@iu.edu
logellis@iu.edu, lomwean@iu.edu
lrborows@iu.edu, cmoverfe@iu.edu
arkali@iu.edu, masoroki@iu.edu
abelani@iu.edu, teswank@iu.edu
aydclink@iu.edu, nvoneper@iu.edu
ml82@iu.edu, ntorpoco@iu.edu
zaycarey@iu.edu, peterabl@iu.edu
amgandhi@iu.edu, animerch@iu.edu
tkalale@iu.edu, alexpan@iu.edu